Facade

Facade · One placeOrder call over inventory+payment+shipping · Build with Waqa

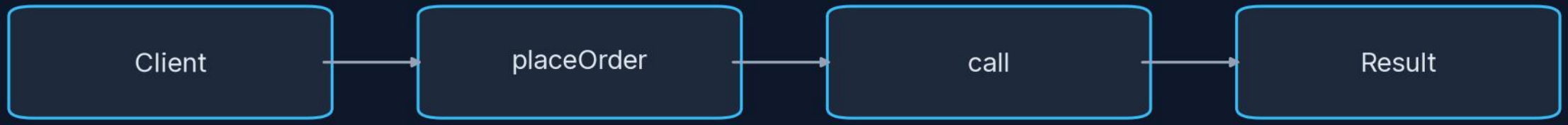
Today's Hook

- Problem: One placeOrder call over inventory+payment+shipping
- Pattern: Facade — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- One placeOrder call over inventory+payment+shipping
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Facade: One placeOrder call over inventory+payment+shippin

Real-World Scenario

- Running example: One placeOrder call over inventory+payment+shipping
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.structural.facade.problem;

import java.util.*;

public final class NaiveCheckout {
  public static String place(String sku, int cents) {
    // client calls three services itself
    boolean inv = true; boolean pay = cents > 0; boolean ship = true;
    if (!inv || !pay || !ship) return "FAIL";
    return "OK:"+sku;
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.structural.facade.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;

final class Inventory { boolean reserve(String sku){ return sku!=null && !sku.isBlank(); } void release(String sku){} }
final class Payments { boolean charge(int cents){ return cents>0; } }
final class Shipping { boolean ship(String addr){ return addr!=null; } }
final class OrderFacade {
  private final Inventory inv=new Inventory();
  private final Payments pay=new Payments();
  private final Shipping ship=new Shipping();
  String placeOrder(String sku, int cents, String addr){
    if (!inv.reserve(sku)) return "FAIL:inventory";
    if (!pay.charge(cents)) { inv.release(sku); return "FAIL:payment"; }
    if (!ship.ship(addr)) return "FAIL:shipping";
    return "OK:"+sku;
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.structural.facade.problem;

import java.util.*;
import java.util.concurrent.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Facade: placeOrder over inventory+payment+shipping ===");
    System.out.println("--- naive ---");
System.out.println(NaiveCheckout.place("SKU", 100));
System.out.println("--- facade ---");
var f=new OrderFacade();
System.out.println(f.placeOrder("SKU", 100, "X"));
System.out.println(f.placeOrder("SKU", 0, "X"));
  }
}
```

Walkthrough Output

```
=== Facade: placeOrder over inventory+payment+shipping ===
--- naive ---
OK:SKU
--- facade ---
OK:SKU
FAIL:payment
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Facade.

Related Patterns

- Contrast Facade with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Facade → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.